AI / ML ASSIGNMENTS CODE EXPLAINED

Beginner-friendly theory + code walkthrough in simple Hinglish

Based on the public repository:

https://github.com/rohitkb11/aiml_learning

Goal: understand what each major code block does, why it is used, and the AI/ML theory behind it.

How to use this guide

Read the Aim and Core idea first. Then read each code snippet and its bullets. The code shown is the important original logic from the repository, with repeated plotting/display lines shortened where they perform the same kind of task. Each section explains the main cells/blocks and points out important limitations.

Numbering clarification: The repository does not provide a clean official list called Assignment 1 through Assignment 9. The nine sections here are a learning-order mapping of the nine main code files: Agent.ipynb, bfs.ipynb, e.ipynb, preprocessing.py, regression.py, classification.py, PCA.ipynb, PCA_KNN_Simple.ipynb, and KMeans_Hierarchical_Clustering_Iris.ipynb. The Python files themselves label preprocessing.py as Assignment 1 and regression.py as Assignment 5. This guide numbering is not presented as the repository author's official numbering.

The repository also contains Practical11.ipynb; it is not included in the nine-section scope of this guide.

Contents / file map

- 1. Agent.ipynb - Intelligent agents and PEAS
- 2. bfs.ipynb - Maze/graph search using BFS and DFS
- 3. e.ipynb - Rule-based reasoning: forward and backward chaining
- 4. preprocessing.py - Titanic data cleaning and preprocessing
- 5. regression.py - Linear and polynomial regression
- 6. classification.py - Decision Trees and Gaussian Naive Bayes
- 7. PCA.ipynb - PCA + Support Vector Classifier (SVC)
- 8. PCA_KNN_Simple.ipynb - PCA + K-Nearest Neighbours
- 9. KMeans_Hierarchical_Clustering_Iris.ipynb - K-Means and hierarchical clustering

Dataset files in the repository include Admission_Predict.xls, Titanic-Dataset.xls, housing.xls, Wine.csv, Iris.xls, and car_evaluation.csv. Always check the actual file format and filename before running code.

Section 1. Intelligent Agent and PEAS

Repository file: Agent.ipynb

Aim: Build a college admission chatbot that displays courses/seats, processes admission, and recommends a course.

Core idea: An agent takes an input (percept), chooses an action using rules/data, and acts on its environment. PEAS describes Performance measure, Environment, Actuators, and Sensors.

Cell 0 - Environment data

```
class CollegeEnvironment:
    def __init__(self):
        self.seats = {"Computer": 120, "IT": 120}
        self.placement = {"Computer": 95, "IT": 90}
```

- `class` is a blueprint. `CollegeEnvironment` stores the world/data with which the agent works.
- `self.seats` is a dictionary: course name -> vacant seats. Admission can change this state.
- `self.placement` stores placement values used by the recommendation rule. The notebook does not specify the units, so treat them as scores.

Theory: Environment = the external world/data the agent interacts with. State = its current condition, such as the number of seats left.

Cell 1 - Sensor / input

```
class Sensors:
    def get_input(self):
        return input("Enter your choice: ")
```

- `input()` asks the user to type something. `return` sends that value back to the caller.
- Python `input()` returns text (a string), so menu choices are compared with values like `"1"`, not integer `1`.

Theory: A sensor provides information to an agent. Here the keyboard is a simulated sensor; there is no physical sensor.

Cell 2 - Actuators

```
class Actuators:
    def show_courses(self):
        print("Courses: Computer Engineering, IT Engineering")

    def show_seats(self, environment):
        print("Vacant seats:", environment.seats)

    def confirm_admission(self, course, environment):
        if environment.seats[course] > 0:
            environment.seats[course] -= 1
            print("Admission confirmed in", course)
            print("Remaining seats:", environment.seats)
```

```
        else:
            print("No seats available in", course)
```

- `show_courses()` displays available courses; `show_seats()` reads current seat data from the environment.
- `if environment.seats[course] > 0` checks availability before admitting a student.
- `-= 1` subtracts one seat after successful admission. `else` handles the zero-seat case.

Theory: Actuator means the mechanism used to perform an action. In this console program, printing text and changing the seat dictionary simulate actions.

Cell 3 - Simple reflex agent

```
def act(self, choice):
    if choice == "1":
        self.actuators.show_courses()
    elif choice == "2":
        self.actuators.show_seats(self.environment)
    elif choice == "3":
        print("Admission confirmed in Computer")
    elif choice == "4":
        print("Admission confirmed in IT")
    elif choice == "5":
        print("Recommended: Computer")
    else:
        print("Invalid choice")
```

- `if/elif` maps each current menu input to a predefined action. `else` handles invalid values.
- Option 1 shows courses; 2 shows seats; 3 and 4 only print a confirmation; 5 always recommends Computer.

Important limitation: Options 3 and 4 in SimpleReflexAgent do not call `confirm_admission()`: they do not check or decrement seats. This is a limitation/bug in the source code, not a real admission transaction.

Theory: A simple reflex agent selects an action from the current percept using condition-action rules. It does not learn from data.

Cell 4 - Model-based agent and recommendation

```
def admission(self, course):
    self.actuators.confirm_admission(course, self.environment)

def recommend_course(self):
    seats = self.environment.seats
    placement = self.environment.placement

    if seats["Computer"] == 0 and seats["IT"] == 0:
        print("No seats available")
    elif seats["Computer"] == 0:
        print("Recommended: IT")
    elif seats["IT"] == 0:
        print("Recommended: Computer")
    else:
        score_comp = seats["Computer"] * 0.3 + placement["Computer"] * 0.7
        score_it = seats["IT"] * 0.3 + placement["IT"] * 0.7
        if score_comp >= score_it:
            print("Recommended: Computer")
```

```
    else:
        print("Recommended: IT")
```

- `admission()` reuses the actuator method, so there is one place for seat checking and updating.
- The recommendation first checks whether either course has no seats. If both have seats, it creates a weighted score.
- Computer score = 30% seat count + 70% placement value; IT uses the same formula. The larger score wins.

Theory: Weighted scoring compares options using several factors with chosen importance weights. The 0.3/0.7 weights are design choices in this demo, not learned values or a validated college policy.

Cell 5 - Starting and running the agent

```
print("Performance Measure: Correct admission and recommendations")
environment = CollegeEnvironment()
sensors = Sensors()
actuators = Actuators()

agent_choice = input("Choose agent (1/2): ")
if agent_choice == "1":
    agent = SimpleReflexAgent(environment, actuators)
elif agent_choice == "2":
    agent = ModelBasedAgent(environment, actuators)
else:
    print("Invalid choice")
    exit()

while True:
    choice = sensors.get_input()
    if choice == "0":
        print("Thank you!")
        break
    agent.act(choice)
```

- Objects are created from the classes. The selected object is stored in `agent`.
- `while True` repeatedly shows/accepts menu input; `break` stops the loop when choice is `0`.
- `agent.act(choice)` calls the selected agent's own `act()` method.

PEAS for this program: P: correct admissions/recommendations; E: courses, seats, placement data; A: display information and update seats; S: user menu input. Printing the PEAS labels describes the design, but the program does not automatically measure success.

Section 2. Maze and Graph Search: BFS vs DFS

Repository file: bfs.ipynb

Aim: Find a route between start and goal in a maze and in a labelled graph, then compare explored nodes and runtime.

Core idea: A search problem has states/nodes, legal moves/edges, a start, and a goal. BFS uses a FIFO queue and explores level by level; DFS uses a LIFO stack and dives down a branch.

Cell 0-1 - Maze and graph

```
maze = [
    "S.....",
    ".####.",
    ".....",
    ".####.",
    "E....."
]

for r in range(len(maze)):
    for c in range(len(maze[0])):
        if maze[r][c] == 'S':
            start = (r, c)
        elif maze[r][c] == 'E':
            end = (r, c)

graph = {
    'A': ['B', 'C'], 'B': ['A', 'D'],
    'C': ['A', 'E'], 'D': ['B', 'F'],
    'E': ['C', 'F'], 'F': ['D', 'E']
}
graph_start = 'A'
graph_end = 'F'
```

- Maze characters: `S` start, `E` end, `#` wall, `.` open cell. Coordinates are `(row, column)`.
- The graph dictionary stores each node and its neighbours. For example, A connects to B and C.

Theory: A graph has vertices (nodes) and edges (connections). A maze is another search space where each open cell is a state.

Cell 2 - Find neighbours

```
def get_neighbours(node):
    if mode == "graph":
        return graph[node]

    r, c = node
    dx = [-1, 1, 0, 0]
    dy = [0, 0, 1, -1]
    results = []

    for i in range(4):
        newR = r + dx[i]
        newC = c + dy[i]
```

```
        if 0 <= newR < len(maze) and 0 <= newC < len(maze[0]):
            if maze[newR][newC] != "#":
                results.append((newR, newC))
    return results
```

- Graph mode returns the listed neighbours. Maze mode calculates up, down, right, and left coordinates.
- The bounds checks stop movement outside the grid; the `!= "#"` check prevents walking through a wall.

Cell 3 - BFS

```
def bfs(start, end):
    queue = [[start]]
    visited = [start]
    explored = 0

    while len(queue) > 0:
        path = queue.pop(0)
        node = path[-1]
        explored = explored + 1

        if node == end:
            return path, explored

        for n in get_neighbours(node):
            if n not in visited:
                queue.append(path + [n])
                visited.append(n)
    return None, explored
```

- Queue starts with one path containing the start. `pop(0)` removes the oldest path (FIFO).
- `path[-1]` is the current node. If it equals the goal, the function returns the route and explored count.
- Each unseen neighbour is appended as a new path (`path + [n]`) and added to `visited` so it is not repeatedly queued.

Theory: BFS finds a shortest path when every move has equal cost because it examines paths in increasing number of edges. A Python `deque` is usually more efficient than repeatedly using `list.pop(0)`.

Cell 4 - DFS

```
def dfs(start, end):
    stack = [[start]]
    visited = [start]
    explored = 0

    while len(stack) > 0:
        path = stack.pop()
        node = path[-1]
        explored = explored + 1

        if node == end:
            return path, explored

        for n in get_neighbours(node):
            if n not in visited:
                stack.append(path + [n])
                visited.append(n)
    return None, explored
```

- `stack.pop()` removes the newest path (LIFO), so the newest branch is explored first.
- DFS may find a path quickly, but it does not guarantee the shortest path.

Memory cue: BFS = FIFO queue (first in, first out). DFS = LIFO stack (last in, first out).

Cell 5-8 - Compare, draw, switch modes

```
def comparision(start, end):
    t1 = time.perf_counter()
    bfs_path, bfs_nodes = bfs(start, end)
    bfs_time = time.perf_counter() - t1

    t2 = time.perf_counter()
    dfs_path, dfs_nodes = dfs(start, end)
    dfs_time = time.perf_counter() - t2

    print("BFS path:", bfs_path, "nodes:", bfs_nodes, "time:", bfs_time)
    print("DFS path:", dfs_path, "nodes:", dfs_nodes, "time:", dfs_time)
    return bfs_path, bfs_nodes, dfs_path, dfs_nodes

mode = "maze"
bfs_path, bfs_nodes, dfs_path, dfs_nodes = comparision(start, end)
show_path(bfs_path)
show_path(dfs_path)

mode = "graph"
bfs_path, bfs_nodes, dfs_path, dfs_nodes = comparision(graph_start, graph_end)
```

- `time.perf_counter()` measures elapsed runtime. The returned tuple contains both algorithms' paths and explored counts.
- `show_path()` copies the maze, replaces `.` cells on the route with `*`, and prints the rows.
- Changing `mode` changes `get_neighbours()` from grid coordinates to graph node labels.

Code note: The final graph cell ends with the bare expression `dfs_path`. In Jupyter, the last expression is displayed; in a normal Python script, it will not print by itself. Use `print(dfs_path)` for consistent output.

Section 3. Rule-Based Reasoning: Forward and Backward Chaining

Repository file: e.ipynb

Aim: Represent simple symptom rules and infer whether the goal `see_doctor` can be proved from the starting facts.

Core idea: A rule-based expert system uses facts plus IF-condition-THEN-conclusion rules. Forward chaining starts from known facts; backward chaining starts from a goal and asks what facts/rules would prove it.

Cell 0 - Rules and facts

```
rules = [
    (("AND", ["fever", "cough"]), "flu"),
    (("AND", ["flu", "body_pain"]), "see_doctor"),
    (("OR", ["sneezing", "runny_nose"]), "cold"),
    (("AND", ["fever", ("OR", ["cough", "sneezing"])]), "infection")
]
facts = ["fever", "cough", "body_pain"]
```

- Each rule is `(condition, conclusion)`. A condition may be a string/fact or a nested `AND`/`OR` group.
- Facts are what the system currently knows. In this demo they are fever, cough, and body pain.
- `show_condition()` recursively turns a nested condition into readable text for printing.

Theory: A fact is a known statement. A production rule says: IF a condition is satisfied, THEN infer a new fact. This is a toy reasoning example, not a medical diagnosis tool.

Cell 1 - Evaluate AND/OR conditions

```
def condition_is_true(condition, known):
    if isinstance(condition, str):
        return condition in known
    operator, parts = condition

    if operator == "AND":
        return all(condition_is_true(part, known) for part in parts)
    elif operator == "OR":
        return any(condition_is_true(part, known) for part in parts)
    return False
```

- If condition is a string, checks whether it is in the current known-facts list.
- `all(...)` means every part must be true (AND). `any(...)` means at least one part must be true (OR).
- The function calls itself for nested conditions; this is recursion.

Cell 1-2 - Forward chaining

```
def forward_chaining(goal, rules, facts):
    known = list(facts)
    change = True
    while change:
        change = False
```

```
        for condition, conclusion in rules:
            if condition_is_true(condition, known) and conclusion not in known:
                known.append(conclusion)
                print("RULES FIRED:", show_condition(condition), "->", conclusion)
                change = False
    print("FACTS:", known)
    return goal in known

forward_chaining("see_doctor", rules, facts)
```

- Copies the input facts into `known`, checks rules, and adds a conclusion whenever its condition is satisfied.
- The goal is true if it appears in the final `known` facts.

Important source issue: After adding a fact, the code sets `change = False` instead of `True`. That can stop repeated passes too early if a later rule would enable an earlier rule. Correct fixed-point chaining usually sets `change = True` whenever a new fact is added.

Cell 3-4 - Backward chaining

```
def prove_condition(condition, facts, rules, checking):
    if isinstance(condition, str):
        return backward_chaining(condition, facts, rules, checking)
    operator, parts = condition
    if operator == "AND":
        return all(prove_condition(p, facts, rules, checking) for p in parts)
    elif operator == "OR":
        return any(prove_condition(p, facts, rules, checking) for p in parts)
    return False

def backward_chaining(goal, facts, rules, checking=None):
    if checking is None:
        checking = set()
    if goal in facts:
        print("Fact:", goal)
        return True
    if goal in checking:
        return False
    checking = checking | {goal}
    for condition, conclusion in rules:
        if conclusion == goal:
            print("To prove:", goal, ", check:", show_condition(condition))
            if prove_condition(condition, facts, rules, checking):
                return True
    return False
```

- Start with a goal, e.g. `see_doctor`; find rules that can conclude that goal; then prove their conditions recursively.
- If the goal is already a fact, return `True`. `checking` avoids repeatedly trying the same goal in a recursive loop.

Difference: Forward chaining: facts -> derive new facts -> check goal. Backward chaining: goal -> search supporting rule -> prove the rule conditions.

Section 4. Data Preprocessing and Exploration

Repository file: preprocessing.py (source header: Assignment 1)

Aim: Clean and explore the Titanic dataset, encode categories, scale numeric features, and divide data into training and testing sets.

Core idea: Raw data usually has missing values, text categories, duplicate/irrelevant columns, and features on different scales. Preprocessing makes data usable for ML models.

Step 2-3 - Load and inspect data

```
df = pd.read_csv("Titanic-Dataset.xls")
print(df.head())
print("Samples:", df.shape[0])
print("Features:", df.shape[1])

for col in df.columns:
   print(col, df[col].count(), df[col].dtype)

df.drop(columns=["Name", "PassengerId"], inplace=True)
print(df.describe())
```

- `df` is a pandas DataFrame (table). `head()` shows first rows; `shape` gives rows and columns.
- `count()` counts non-missing values; `dtype` shows the column type. `describe()` summarizes numeric columns.
- `drop(..., inplace=True)` removes columns directly from `df`; the code removes name and passenger ID as identifiers rather than useful modelling inputs.

File-format check: `read_csv()` expects CSV text. The repository name ends in `.xls`. If the file is a real Excel workbook, use `pd.read_excel(...)`; if it is CSV text saved with an `.xls` extension, `read_csv` may work. Confirm the actual file format.

Step 4 - Missing values

```
for col in df.columns:
   print(col, df[col].isnull().sum())

df.drop(columns=["Cabin"], inplace=True)
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
print(df.isnull().sum())
```

- `isnull().sum()` counts missing values per column.
- Cabin is dropped because it has many missing values. Age uses median; Embarked uses most frequent category (mode).

Theory: Median is less sensitive to extreme numeric values than the mean. Mode is useful for categorical columns. Imputation fills gaps instead of deleting every incomplete row.

Step 5 - Encode text categories

```
encoder = LabelEncoder()
df["Sex"] = encoder.fit_transform(df["Sex"])

df = pd.get_dummies(df, columns=["Embarked"], dtype=int)
print(df.head())
```

- `LabelEncoder` maps categories in Sex to integer codes.
- `get_dummies` one-hot encodes Embarked by creating one 0/1 column for each category.

Theory: Many ML algorithms expect numeric inputs. For unordered categories, one-hot encoding avoids implying that one category is numerically greater than another.

Steps 6-7 - Summary and plots

```
print(df.describe())
print(df["Survived"].value_counts())
print(df.groupby("Pclass")["Survived"].mean())

sns.barplot(data=df, x="Pclass", y="Survived", hue="Sex")
sns.boxplot(x=df["Age"])
sns.boxplot(x=df["Fare"])
plt.hist(df["Age"])
plt.show()
```

- `value_counts()` counts each survival class. `groupby(...).mean()` estimates survival rate within each passenger class because Survived is 0/1.
- Bar plot compares rates; boxplots show spread/outliers; histogram shows the distribution of Age. These plots help spot patterns before modelling.

Steps 8-10 - Scaling and split

```
scaler = MinMaxScaler()
cols_to_scale = ["Age", "Fare"]
df[cols_to_scale] = scaler.fit_transform(df[cols_to_scale])

X = df.drop(columns=["Survived"])
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)
```

- MinMaxScaler maps each selected column to a 0-1 range using its observed minimum and maximum.
- `X` contains input features; `y` is the target Survived. `test_size=0.2` keeps 20% for testing; `random_state=42` makes the split repeatable.

Best-practice note: The source scales the full dataset before splitting. In a stricter ML workflow, split first, fit the scaler only on `X_train`, then transform `X_train` and `X_test`; otherwise test-set statistics influence preprocessing (data leakage).

Section 5. Linear and Polynomial Regression

Repository file: regression.py (source header: Assignment 5)

Aim: Predict median house value and compare a straight-line model with a degree-2 polynomial model.

Core idea: Regression predicts a numeric value. Linear regression fits a linear relationship; polynomial regression adds squared and interaction features so a linear model can represent a curved surface.

Steps 2-3 - Load, clean, find correlated features

```
df = pd.read_csv("housing.xls")
df.drop(columns=["longitude", "latitude"], inplace=True)
df["total_bedrooms"] = df["total_bedrooms"].fillna(df["total_bedrooms"].median())
df = pd.get_dummies(df, columns=["ocean_proximity"], dtype=int)
output_feature = "median_house_value"

correlation_matrix = df.corr(numeric_only=True)[[output_feature]].sort_values(
   by=output_feature, ascending=False
)
top_feature = correlation_matrix.index[1]
secnd_top_feature = correlation_matrix.index[2]
```

- Missing bedroom counts are filled with the median; ocean proximity text is converted to dummy columns.
- Correlation gives a value between -1 and +1 for linear association. The target itself is normally first with correlation 1, so index 1 and 2 select the next two entries after sorting.

File-format check: As in the preprocessing script, check `housing.xls`: use `read_excel` for a true Excel file and `read_csv` for CSV text. The source removes longitude/latitude, so those features are not used in this model.

Step 4 - Train linear regression

```
X = df[[top_feature]]
y = df[output_feature]
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=RANDOM_STATE
)
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
y_train_pred = linear_model.predict(X_train)
y_test_pred = linear_model.predict(X_test)
```

- `X` has one chosen input feature; `y` contains median house values. The split protects a holdout set for evaluation.
- `fit()` learns model coefficients from training data. `predict()` estimates values for training and test records.
- Coefficient is the estimated change in house value per one-unit change of the feature; intercept is the baseline when the feature is zero.

Step 4 - Metrics

```
mse = mean_squared_error(y_test, y_test_pred)
mae = mean_absolute_error(y_test, y_test_pred)
```

```
r2 = r2_score(y_test, y_test_pred)
print("MSE:", mse)
print("MAE:", mae)
print("R2 Score:", r2)
```

- MSE = average squared error; large mistakes receive extra penalty. Lower is better.
- MAE = average absolute error in the target's units. Lower is better.
- R-squared measures how much target variation is explained compared with a mean baseline; closer to 1 is generally better, and it can be negative.

Step 6 - Polynomial regression

```
X_poly_data = df[[top_feature, secnd_top_feature]]
y_poly_data = df[output_feature]
X_train_poly_data, X_test_poly_data, y_train_poly, y_test_poly = train_test_split(
   X_poly_data, y_poly_data, test_size=0.2, random_state=RANDOM_STATE
)
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train_poly_data)
X_test_poly = poly.transform(X_test_poly_data)
poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train_poly)
y_test_pred_poly = poly_model.predict(X_test_poly)
```

- Two selected inputs are expanded into first-order, squared, and pairwise interaction terms (degree 2).
- `fit_transform` learns and transforms the training features; `transform` applies the same feature mapping to test data.
- The model is still LinearRegression, but it can represent curves because the inputs now include polynomial terms.

Steps 7-10 - Surface plot, user prediction, comparison

```
xx1, xx2 = np.meshgrid(
   np.linspace(x1_min, x1_max, 50),
   np.linspace(x2_min, x2_max, 50)
)
grid_points = np.c_[xx1.ravel(), xx2.ravel()]
grid_points_poly = poly.transform(grid_points)
yy_pred = poly_model.predict(grid_points_poly).reshape(xx1.shape)

user_input_1 = float(input(f"Enter {top_feature}: "))
user_input_2 = float(input(f"Enter {secnd_top_feature}: "))
user_data = pd.DataFrame({top_feature: [user_input_1], secnd_top_feature: [user_input_2]})
prediction = poly_model.predict(poly.transform(user_data))
```

- Meshgrid creates a grid of feature combinations so the polynomial model's predicted surface can be drawn in 3D.
- User inputs are converted to floats, put into a one-row DataFrame, transformed using the same polynomial mapping, and passed to `predict()`.
- The final table and bar charts compare training/test MSE, MAE, and R2 for both models. Prefer good test performance, not just training fit.

Code note: Feature selection by correlation is performed before splitting the data. For a strict evaluation, choose features using only training data or inside cross-validation to avoid leaking test-set information into the modelling choice.

Section 6. Classification: Decision Tree and Naive Bayes

Repository file: classification.py

Aim: Classify admission chance as Low or High using student information and compare Decision Trees with Gaussian Naive Bayes.

Core idea: Classification predicts a category rather than a continuous number. The source converts `Chance of Admit` into a binary target, trains three models, evaluates them, and predicts a new student.

Step 1 - Prepare target and features

```
df = pd.read_csv("Admission_Predict.xls")
df.columns = df.columns.str.strip()
df = df.drop(columns=["Serial No."])
threshold = df["Chance of Admit"].quantile(0.75)
df["Chance of Admit"] = (df["Chance of Admit"] >= threshold).astype(int)
X = df.drop(columns=["Chance of Admit"])
y = df["Chance of Admit"]
```

- Column names are stripped of extra spaces; Serial No. is an identifier, not a useful predictive feature.
- 75th percentile is used as a threshold: values at or above it become class 1; others become class 0. `CLASS_NAMES` maps 0 to Low and 1 to High.
- `X` = student features; `y` = binary target label.

File-format check: The code uses `read_csv` for `Admission_Predict.xls`. If it is a true Excel file, change to `pd.read_excel`; if CSV text is stored under `.xls`, `read_csv` may work.

Step 2 - Train/test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE,
    stratify=y
)
```

- 80% is training and 20% is testing because `TEST_SIZE = 0.20`.
- `stratify=y` tries to preserve the Low/High proportion in both sets; `random_state` makes results repeatable.

Steps 3-4 - Evaluation and confusion matrix

```
def evaluate_model(model, model_name):
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)
    train_metrics = {
        "Accuracy": accuracy_score(y_train, y_train_pred),
        "Precision": precision_score(y_train, y_train_pred, zero_division=0),
        "Recall": recall_score(y_train, y_train_pred, zero_division=0),
        "F1 Score": f1_score(y_train, y_train_pred, zero_division=0)
    }
    test_metrics = {
        "Accuracy": accuracy_score(y_test, y_test_pred),
        "Precision": precision_score(y_test, y_test_pred, zero_division=0),
        "Recall": recall_score(y_test, y_test_pred, zero_division=0),
        "F1 Score": f1_score(y_test, y_test_pred, zero_division=0)
```

```
    }
    return train_metrics, test_metrics, y_test_pred

def plot_confusion_matrix(y_true, y_pred, title):
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(cm, annot=True, fmt="d", xticklabels=CLASS_NAMES,
                yticklabels=CLASS_NAMES)
```

- Accuracy = fraction correct; precision = how many predicted positives were positive; recall = how many actual positives were found; F1 balances precision and recall.
- `zero_division=0` avoids a metric warning and returns 0 if the score's denominator is zero.
- Confusion matrix compares actual vs predicted classes; heatmap makes correct and incorrect predictions easier to see.

Steps 5-7 - Three classifiers

```
dt_entropy = DecisionTreeClassifier(
    criterion="entropy", max_depth=4, min_samples_split=10,
    min_samples_leaf=5, random_state=RANDOM_STATE
)
dt_entropy.fit(X_train, y_train)

dt_gini = DecisionTreeClassifier(
    criterion="gini", max_depth=4, min_samples_split=10,
    min_samples_leaf=5, random_state=RANDOM_STATE
)
dt_gini.fit(X_train, y_train)

nb = GaussianNB(priors=None, var_smoothing=1e-9)
nb.fit(X_train, y_train)
```

- Decision Tree recursively splits data into groups. Entropy and Gini are two ways to measure how mixed a node's classes are.
- `max_depth=4` limits tree depth; `min_samples_split=10` and `min_samples_leaf=5` discourage tiny splits/leaves.
- Gaussian Naive Bayes uses Bayes' theorem and assumes each feature's values follow a Gaussian distribution within each class, with conditional independence between features.

Step 8-9 - Compare and predict for a new student

```
results = pd.DataFrame(
    [entropy_test, gini_test, nb_test],
    index=["Entropy", "Gini", "Naive Bayes"]
)
results[["Accuracy", "Precision", "Recall", "F1 Score"]].plot(kind="bar")

student = pd.DataFrame([{
    "GRE Score": gre, "TOEFL Score": toefl,
    "University Rating": university_rating, "SOP": sop,
    "LOR": lor, "CGPA": cgpa, "Research": research
}])
predictions = {
    "Decision Tree (Entropy)": dt_entropy.predict(student)[0],
    "Decision Tree (Gini)": dt_gini.predict(student)[0],
    "Naive Bayes": nb.predict(student)[0]
}
```

- Bar chart compares test metrics across all three methods; scores close to 1 are generally better for these metrics.

- `predict_student()` prompts for GRE, TOEFL, rating, SOP, LOR, CGPA, and research, validates their ranges, and formats the values in a DataFrame with matching feature names.
- Each model gives 0 or 1, then `CLASS_NAMES[value]` converts that number to Low/High.

Interpretation warning: “High” means the student’s admission chance is at or above the dataset’s 75th-percentile threshold. It does not mean admission is guaranteed.

Section 7. Principal Component Analysis (PCA) + SVC

Repository file: PCA.ipynb

Aim: Scale Wine dataset features, train a Support Vector Classifier, reduce dimensions to two principal components, and visualize/evaluate classification.

Core idea: PCA is an unsupervised dimensionality-reduction method. It rotates the feature space into principal components that preserve as much variance as possible. SVC is a supervised classifier.

Cells 4-9 - Load, check, split X and y

```
df = pd.read_csv("Wine.csv")
print(df.shape)
print(df["Customer_Segment"].unique())
print(df.isnull().sum())
print(df.duplicated().sum())

X = df.drop("Customer_Segment", axis=1)
y = df["Customer_Segment"]
```

- `shape` gives sample/feature count. `unique()` lists target categories; `isnull()` checks missing values; `duplicated()` counts duplicate rows.
- Target column `Customer_Segment` is set aside as `y`; all remaining columns become input `X`.

Cells 11-14 - Standardize and baseline SVC

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
model = SVC()
model.fit(X_train, y_train)
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
print(accuracy_score(y_train, y_train_pred))
print(accuracy_score(y_test, y_test_pred))
cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cm, annot=True, fmt="d")
```

- StandardScaler changes each feature to roughly mean 0 and standard deviation 1. Scaling matters for distance/margin-based algorithms like SVC.
- `fit()` trains the classifier; `predict()` makes class guesses; accuracy and confusion matrix measure predictions.

Code quality note: The notebook fits the scaler on all rows before the train/test split. To avoid leakage, split first, fit scaler on training rows only, and use `transform()` on test rows.

Cells 16-20 - PCA and explained variance

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
pca_df["quality"] = y.values

X = pca_df[["PC1", "PC2"]]
y = pca_df["quality"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(pca.explained_variance_ratio_)
```

- `n_components=2` compresses the original many features into PC1 and PC2, which are new numeric combinations of old features.
- `explained_variance_ratio_` reports the fraction of the data variation captured by each component. Their sum tells how much variance the two components retain.
- The notebook plots original features vs PC1/PC2 and also plots explained variance from PCA fit using all components.

Naming issue: The source originally calls the target `Customer_Segment`, but the PCA table renames it to `quality`. This is confusing; use one consistent target name so it is clear the label is still Customer_Segment.

Cells 21-25 - SVC after PCA

```
model = SVC(kernel="rbf")
model.fit(X_train, y_train)
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
print("Training Accuracy:", accuracy_score(y_train, y_train_pred))
print("Testing Accuracy:", accuracy_score(y_test, y_test_pred))
print(classification_report(y_test, y_test_pred))

plt.scatter(X_test["PC1"], X_test["PC2"], c=y_test_pred, cmap="viridis", alpha=0.7)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.colorbar(label="Predicted Quality")
plt.show()
```

- RBF kernel lets SVC form a nonlinear decision boundary. The report includes precision, recall, and F1 by class.
- The scatter plot places test rows in 2D PCA space, coloured by predicted classes.

Important method note: PCA is also fitted on the full scaled dataset before splitting in this source. A clean evaluation fits PCA on the training data only, then transforms the test data with the same PCA object.

Section 8. PCA + K-Nearest Neighbours (KNN)

Repository file: PCA_KNN_Simple.ipynb

Aim: Compare KNN classification on original standardized features and after reducing the Wine dataset to two principal components.

Core idea: KNN is an instance-based classifier: it finds the k closest training examples and predicts the majority class among them. Because it depends on distances, feature scaling is important.

Cells 4-12 - Data preparation

```
df = pd.read_csv("Wine.csv")
X = df.drop("Customer_Segment", axis=1)
y = df["Customer_Segment"]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
```

- Same basic preparation as PCA + SVC: separate features/target, standardize, then split train/test.
- `stratify=y` helps keep the segment proportions similar across training and testing.

Leakage note: The scaler is fitted before splitting in the repository notebook. Best practice is to fit the scaler on training data only.

Cells 13-14 - KNN baseline

```
model = KNeighborsClassifier()
model.fit(X_train, y_train)
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
print("Training Accuracy:", accuracy_score(y_train, y_train_pred))
print("Testing Accuracy:", accuracy_score(y_test, y_test_pred))
print(classification_report(y_test, y_test_pred))
cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cm, annot=True, fmt="d")
```

- `fit()` stores the training examples; KNN does most of its work at prediction time by comparing distances to those examples.
- With no argument, scikit-learn's KNeighborsClassifier uses its default neighbour count (normally k=5). The later PCA model explicitly uses `n_neighbors=5`.

Cells 16-23 - PCA / variance / feature reduction

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
pca_df["quality"] = y.values

cumulative_variance = np.cumsum(pca_full.explained_variance_ratio_)
print("Components needed for 95% variance:",
```

```
    np.argmax(cumulative_variance >= 0.95) + 1)

n_before = X_scaled.shape[1]
n_after = X_pca.shape[1]
print("Before PCA:", n_before, "After PCA:", n_after)
```

- `np.cumsum()` adds explained-variance ratios cumulatively. `argmax(condition)+1` finds the first component count reaching 95%.
- The notebook displays feature count before/after PCA and charts how old features are compressed into PC1 and PC2.

Source detail: The target variable is named `Customer_Segment` at first but is stored in a new PCA DataFrame column called `quality`. This is a naming inconsistency, not a new target.

Cells 24-28 - KNN on PCA components

```
X = pca_df[["PC1", "PC2"]]
y = pca_df["quality"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_test_pred = model.predict(X_test)
print(accuracy_score(y_test, y_test_pred))
plt.scatter(X_test["PC1"], X_test["PC2"], c=y_test_pred, cmap="viridis")
```

- After PCA, KNN only uses PC1 and PC2 distances. This is simpler and easier to plot, but some information may have been lost.
- Compare test accuracy and classification report before vs after PCA to see the trade-off between fewer features and predictive performance.

Evaluation note: As in the PCA notebook, this code fits PCA before train/test split. For fair evaluation, fit scaler and PCA on training data only, then transform testing data.

Section 9. K-Means and Hierarchical Clustering

Repository file: KMeans_Hierarchical_Clustering_Iris.ipynb

Aim: Cluster Iris flowers using numeric measurements, choose K with an elbow plot, build a hierarchical dendrogram, and compare cluster quality metrics.

Core idea: Clustering is unsupervised learning: the algorithm groups similar rows without using the Species label to form the clusters. Species is retained afterward only to analyse how clusters relate to known labels.

Cells 2-6 - Setup and data

```
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
RANDOM_STATE = 42
K = 3

df = pd.read_csv("Iris.csv")
species = df["Species"].copy()
X = df.drop(columns=["Id", "Species"])
print("Features used for clustering:", X.columns.tolist())
```

- Thread settings limit some numerical-library threading. `RANDOM_STATE` makes random splits repeatable; `K=3` fixes the chosen cluster count for comparison.
- `Species` is saved separately and removed from X so clustering uses only measurements, not the answer label. `Id` is also removed.

Repository filename issue: The repository root lists `Iris.xls`, but this notebook reads `Iris.csv`. Unless an `Iris.csv` file exists separately or is created locally, this line will raise FileNotFoundError. Change the filename or supply the expected CSV.

Cell 8 - Train/test split and scaling

```
all_indices = np.arange(len(X))
train_indices, test_indices = train_test_split(
    all_indices, test_size=0.20, random_state=RANDOM_STATE
)
X_train = X.iloc[train_indices]
X_test = X.iloc[test_indices]
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
X_scaled = scaler.transform(X)
```

- An index split picks 80% rows for training and 20% for testing; `iloc` selects those rows.
- The scaler is fitted only on training features, then applied to test and all data. This is the right pattern for keeping test statistics out of the fit.

Cell 10 - Cluster evaluation helpers

```
def calculate_wcss(X_data, labels):
    wcss = 0
    for c in np.unique(labels):
```

```
        points = X_data[labels == c]
        centroid = points.mean(axis=0)
        wcss += np.sum((points - centroid) ** 2)
    return wcss

def calculate_gini(labels, true_labels):
    rows = []
    for c in np.unique(labels):
        p = pd.Series(np.asarray(true_labels)[labels == c]).value_counts(normalize=True)
        rows.append({"Cluster": c, "Size": int((labels == c).sum()),
                     "Majority Species": p.idxmax(), "Gini": 1 - np.sum(p ** 2)})
    table = pd.DataFrame(rows).set_index("Cluster")
    weighted_gini = np.sum(table["Size"] * table["Gini"]) / table["Size"].sum()
    return table, weighted_gini

def dunn_index(X_data, labels):
    distances = pairwise_distances(X_data, metric="euclidean")
    clusters = np.unique(labels)
    max_intra = max(
        distances[np.ix_(labels == c, labels == c)].max() for c in clusters
    )
    min_inter = min(
        distances[np.ix_(labels == a, labels == b)].min()
        for i, a in enumerate(clusters) for b in clusters[i + 1:]
    )
    return min_inter / max_intra
```

- WCSS sums squared distances of each point from its own cluster centroid; lower means more compact clusters.
- Gini is used here as a cluster-purity score using known Species after clustering; lower Gini means each cluster contains fewer mixed species.
- Dunn index divides the smallest between-cluster distance by the largest within-cluster diameter; higher generally means better separation.

Theory: Because clustering is unsupervised, Species does not train the clustering algorithm; it is used afterward to evaluate the grouping. Gini here is an external evaluation measure, not the same role as Gini impurity in a classification decision tree.

Cells 16-17 - Elbow method for K-Means

```
k_values = np.arange(1, 11)
elbow_wcss = []
for k in k_values:
    model = KMeans(n_clusters=int(k), n_init=10,
                   random_state=RANDOM_STATE, algorithm="lloyd")
    model.fit(X_scaled)
    elbow_wcss.append(model.inertia_)

# Plot K against WCSS; choose the bend / elbow in the curve
```

- The code tries K=1 through K=10 and records `inertia_`, the model's within-cluster squared-distance sum.
- As K increases, WCSS usually decreases. The elbow is where adding more clusters gives a smaller improvement; source notes choose K=3.

Source detail: The elbow search fits on `X_scaled` (all rows), while the final K-Means below is fitted on training rows. The notebook uses the all-data elbow as a K-selection aid.

Cell 19 - Fit K-Means

```
kmeans = KMeans(
    n_clusters=K, init="k-means++", n_init=10,
    max_iter=300, random_state=RANDOM_STATE, algorithm="lloyd"
)
kmeans.fit(X_train_scaled)
train_labels = kmeans.predict(X_train_scaled)
test_labels = kmeans.predict(X_test_scaled)
```

- K-Means initializes K centroids, assigns points to the nearest centroid, recalculates means, and repeats until stable/iteration limit.
- `k-means++` chooses starting centres more carefully; `n_init=10` tries multiple initializations; `max_iter=300` caps iterations.
- The model is fitted on training data; test examples are assigned to the nearest learned centre.

Cells 28-36 - Agglomerative clustering and dendrogram

```
agglo = AgglomerativeClustering(
    n_clusters=K, metric="euclidean", linkage="ward"
)
agglo_raw_labels = agglo.fit_predict(X_scaled)

linkage_matrix = linkage(X_scaled, method="ward")
dendro = dendrogram(linkage_matrix)
plt.axhline(cut_height, color="red", linestyle="--")
```

- Agglomerative clustering starts with each point as its own cluster, then repeatedly merges the closest clusters until K remain.
- Ward linkage merges clusters to keep the increase in within-cluster variation small. The dendrogram shows which clusters merge and at what distance.

Consistency issue: The notebook splits and scales train/test data, but `AgglomerativeClustering` and `linkage()` are fitted on full `X_scaled`, including test rows. That is inconsistent with the K-Means train-only fit and should be acknowledged when comparing results.

Cells 38-45 - Compare the methods

```
comparison = pd.DataFrame([
    {"Method": name, "Clusters": K,
     "WCSS": calculate_wcss(X_scaled, labels),
     "Weighted Gini": calculate_gini(labels, species)[1],
     "Dunn Index": dunn_index(X_scaled, labels)}
    for name, labels in all_labels.items()
]).set_index("Method")

print("Most compact:", comparison["WCSS"].idxmin())
print("Purest clusters:", comparison["Weighted Gini"].idxmin())
print("Best separated:", comparison["Dunn Index"].idxmax())
```

- The comparison table combines WCSS, weighted Gini, and Dunn Index for K-Means and hierarchical results.
- For these metrics: lower WCSS = more compact; lower weighted Gini = purer according to Species; higher Dunn = better separated.
- Visualizations show actual species beside the discovered clusters and plot cluster centroids. Cluster IDs themselves are arbitrary labels.

Important nuance: The source aligns numeric cluster IDs to species names/colors after clustering using a best-match mapping. This does not create the clusters, but it uses true labels for presentation/evaluation. Do not describe this as fully label-free evaluation.

Final quick revision

PEAS: P = how success is measured; E = where agent works; A = how it acts; S = what information it senses.

BFS vs DFS: BFS uses FIFO queue and finds shortest route in unweighted graphs; DFS uses LIFO stack and does not promise shortest path.

Forward vs backward chaining: Forward starts from facts and derives conclusions; backward starts from a goal and tries to prove it.

Preprocessing: Inspect -> handle missing data -> encode categories -> scale -> split X/y -> split train/test.

Regression: Predicts numeric values. MSE/MAE lower is better; R2 closer to 1 is generally better.

Classification: Predicts classes. Accuracy, precision, recall, F1 and confusion matrix provide different views of performance.

PCA: Reduces feature dimensions into principal components. Explained variance tells how much variability is retained.

KNN: Predicts using the labels of nearby training points; scale features because distances are affected by units.

Clustering: Groups similar examples without training on labels. K-Means uses centroids; agglomerative clustering merges groups and can be shown as a dendrogram.

Important repository-specific checks

- Confirm dataset formats: scripts use pd.read_csv for some files with .xls extensions.
- KMeans_Hierarchical_Clustering_Iris.ipynb reads Iris.csv, but the listed repository file is Iris.xls.
- The simple reflex agent prints admission confirmations without changing seat counts.
- The forward-chaining loop should set its change flag to True when it adds a new fact.
- The PCA notebooks fit the scaler/PCA on all rows before splitting; this can cause data leakage.
- The PCA notebooks call the Customer_Segment target column quality in later cells; keep target naming consistent.
- The clustering notebook fits hierarchical clustering on all rows even though it created a train/test split.

Repository source links

Agent.ipynb - https://github.com/rohitkb11/aiml_learning/blob/master/Agent.ipynb

bfs.ipynb - https://github.com/rohitkb11/aiml_learning/blob/master/bfs.ipynb

e.ipynb - https://github.com/rohitkb11/aiml_learning/blob/master/e.ipynb

preprocessing.py - https://github.com/rohitkb11/aiml_learning/blob/master/preprocessing.py

regression.py - https://github.com/rohitkb11/aiml_learning/blob/master/regression.py

classification.py - https://github.com/rohitkb11/aiml_learning/blob/master/classification.py

PCA.ipynb - https://github.com/rohitkb11/aiml_learning/blob/master/PCA.ipynb

PCA_KNN_Simple.ipynb - https://github.com/rohitkb11/aiml_learning/blob/master/PCA_KNN_Simple.ipynb

KMeans_Hierarchical_Clustering_Iris.ipynb - https://github.com/rohitkb11/aiml_learning/blob/master/KMeans_Hierarchical_Clustering_Iris.ipynb

This guide explains code intent and the methods used in the repository. Predictions and metric values depend on running the notebooks with the correct datasets and installed package versions.